Database Management System
(UCS310)

DBMS Project Synopsis
Sports Recommendation System

Degree / Year	B.E. COPC · Second Year
Course	UCS310 – Database Management Systems
Academic Year	2024–2025
Department	Computer Science & Engineering
Institute	Thapar Institute of Engineering and Technology, Patiala – 147001
Submitted By	Samyak Jain (1024170400) Sangam Sangwan (1024170388)
Batch No.	2Q34
Submitted To	Ms. Gayatri Saxena

1. Introduction

Sports and physical activities are essential for maintaining physical fitness and mental well-being. However, selecting an appropriate sport is non-trivial — it depends on a combination of personal factors including age, body weight, stamina level, available time per session, health conditions, and individual preferences such as indoor vs outdoor environments and team vs individual formats.

This project implements a backend-based **Sports Recommendation System** using SQL and PL/SQL on Oracle Database. The system stores structured user data across normalised relational tables and generates personalised sport recommendations through SQL JOIN queries, database views, stored procedures, functions, and triggers — demonstrating the power of a relational DBMS over flat file-based approaches.

A DBMS-backed approach ensures **data consistency**, **reduced redundancy**, **referential integrity** via foreign keys and CHECK constraints, and **efficient retrieval** through indexed queries — none of which are achievable with simple file-based systems.

2. Problem Statement

Manual sport selection is inherently subjective and inconsistent. A coach or advisor without structured data may overlook critical factors such as a user's stamina ceiling, health-related restrictions, or weight eligibility for contact sports.

Limitations of the existing / manual approach:

- No structured data storage — decisions are made ad-hoc
- Health conditions (e.g. asthma, knee pain) may be ignored
- Age and weight eligibility limits are not systematically enforced
- No reproducibility — the same user may get different advice each time

This project addresses these limitations by building a DBMS-backed recommendation engine where all matching logic is encoded in SQL, ensuring consistent, auditable, and constraint-validated results.

3. Objectives of the Project

- Design a normalised database using Entity-Relationship (ER) modelling.
- Convert the ER model into relational tables with primary and foreign keys.
- Normalise the schema up to Third Normal Form (3NF).
- Implement the schema using SQL DDL (CREATE TABLE with CHECK constraints).
- Insert and manage data using SQL DML (INSERT, UPDATE, DELETE).
- Generate recommendations using multi-table SQL JOIN queries and a database VIEW.
- Implement PL/SQL constructs: stored procedure, function, trigger, explicit cursor, and exception handling.
- Demonstrate transaction control using COMMIT, SAVEPOINT, and ROLLBACK.
- Write analytical queries using aggregate functions, GROUP BY, HAVING, and subqueries.

4. Scope of the Project

The scope is limited to **backend implementation** using SQL and PL/SQL on Oracle Database (Live SQL). No frontend interface or machine learning techniques are included.

Types of Users

- **Admin / Developer** — runs DDL/DML scripts, manages the sports catalogue, and executes PL/SQL procedures.
- **End User** — represented as a data record; receives sport recommendations generated by the stored procedure.

Modules

- User Data Management
- Physical Traits Management
- Lifestyle & Interests Management
- Sport Information Management
- Recommendation Module (VIEW + Stored Procedure)
- Analytics Module (Aggregate & Subquery Queries)

5. Proposed System Description

The system uses five normalised relational tables. SQL JOIN operations compare user attributes against sport requirements across seven matching criteria:

Weight range	User weight_kg $\in$ [min_weight_kg, max_kg_weight]
Age range	User age $\in$ [min_age, max_age]
Time availability	free_time_hours $\geq$ min_time_required_hrs
Stamina level	get_stamina_score(user) $\geq$ get_stamina_score(sport)
Team preference	prefers_team = is_team_sport
Indoor preference	prefers_indoor = 'Yes' $\rightarrow$ sport_type = 'Indoor' (and vice versa)
Health condition	health_issue $\neq$ 'None' $\rightarrow$ only low_impact = 'Yes' sports

A database VIEW (*sport_recommendation_view*) encapsulates the full JOIN logic, providing a clean abstraction layer. A PL/SQL stored procedure iterates over the view results using an explicit cursor and prints formatted recommendations to DBMS_OUTPUT.

6. Database Design

6.1 Entities and Attributes

USERS	user_id (PK), name, age, gender
PHYSICAL_TRAITS	trait_id (PK), user_id (FK), height_cm, weight_kg, stamina_level, flexibility_level
LIFESTYLE	lifestyle_id (PK), user_id (FK), activity_level, free_time_hours, health_issue
INTERESTS	interest_id (PK), user_id (FK), prefers_indoor, prefers_team, competitiveness
SPORTS	sport_id (PK), sport_name, sport_type, required_stamina, is_team_sport, min_weight_kg, max_kg_weight, min_time_required_hrs, low_impact, min_age, max_age, equipment_required

6.2 Relationships

- USERS 1:1 PHYSICAL_TRAITS (one user has exactly one physical profile)
- USERS 1:1 LIFESTYLE (one user has exactly one lifestyle record)
- USERS 1:1 INTERESTS (one user has exactly one interests record)
- USERS N:M SPORTS (derived via the recommendation VIEW — one user may match many sports; one sport may suit many users)

The ER diagram (attached) uses single vertical bars on both ends of each 1:1 relationship line, with a dashed arrow for the derived N:M VIEW relationship.

7. Normalization

Functional Dependencies (sample)

	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Functional Dependency' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Functional Dependency', textColor=Color(.117647,.227451,.372549,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
USERS	`user_id → name, age, gender`
PHYSICAL_TRAITS	`trait_id → user_id, height_cm, weight_kg, stamina_level, flexibility_level`
LIFESTYLE	`lifestyle_id → user_id, activity_level, free_time_hours, health_issue`

INTERESTS	`interest_id → user_id, prefers_indoor, prefers_team, competitiveness`
SPORTS	`sport_id → sport_name, sport_type, required_stamina, is_team_sport, …`

Normal Forms

- **1NF:** All attributes hold atomic values. No repeating groups.
- **2NF:** No partial dependency. Every non-key attribute depends on the full primary key. All tables use single-column surrogate PKs, so 2NF is automatically satisfied.
- **3NF:** No transitive dependencies. User demographics, physical traits, lifestyle, and interests are stored in separate tables — none of them depends on another non-key attribute.

The schema is in 3NF. BCNF is also satisfied since every determinant is a candidate key in all tables.

8. Database Implementation

8.1 SQL Implementation (DDL)

All five tables are created with PRIMARY KEY, FOREIGN KEY, NOT NULL, and **CHECK constraints** on every categorical column, e.g.:

```
CONSTRAINT chk_stamina_level CHECK (stamina_level IN ('Low','Medium','High'))
CONSTRAINT chk_prefers_indoor CHECK (prefers_indoor IN ('Yes','No'))
CONSTRAINT chk_sport_type CHECK (sport_type IN ('Indoor','Outdoor'))
CONSTRAINT chk_weight_range CHECK (min_weight_kg < max_kg_weight)
```

8.2 SQL Implementation (DML)

- **INSERT:** 6 users with diverse profiles; 8 sports covering indoor, outdoor, team, solo, low-impact and high-impact variants.
- **UPDATE:** Rohan Singh's free_time_hours updated to 3 hours.
- **DELETE:** Parameterised deletion of a test user across all child tables before removing the parent USERS row.

8.3 SELECT Queries

■ JOIN Query — Recommendation View

```
SELECT u.name, s.sport_name, s.sport_type
FROM sport_recommendation_view
WHERE user_id = 2;
```

■ Aggregate + GROUP BY — Matches per user

```
SELECT u.name, COUNT(s.sport_id) AS matched_count
FROM users u LEFT JOIN sport_recommendation_view s ON s.user_id = u.user_id
GROUP BY u.user_id, u.name
ORDER BY matched_count DESC;
```

■ GROUP BY + HAVING — Popular sports (>1 eligible user)

```
SELECT sport_name, COUNT(user_id) AS eligible_users
FROM sport_recommendation_view
GROUP BY sport_name
HAVING COUNT(user_id) > 1;
```

■ Subquery — Users above average match count

```
SELECT name, COUNT(sport_id) AS sport_count
FROM sport_recommendation_view
GROUP BY user_id, name
HAVING COUNT(sport_id) > (SELECT AVG(cnt) FROM
(SELECT COUNT(sport_id) cnt FROM sport_recommendation_view GROUP BY user_id));
```

8.4 VIEW

sport_recommendation_view encapsulates the full seven-criteria JOIN logic. It uses the **get_stamina_score()** function for stamina comparison, making the view concise and the function reusable across the schema.

9. PL/SQL Components

9.1 Function — get_stamina_score

★ New in this version

Converts a stamina label ('Low' | 'Medium' | 'High') to a numeric score (1/2/3). Used inside the recommendation VIEW and callable independently. Eliminates repeated CASE blocks, keeping logic DRY.

```
SELECT get_stamina_score('High') FROM dual; -- returns 3
```

9.2 Trigger — trg_validate_user_age

★ New in this version

A BEFORE INSERT OR UPDATE row-level trigger on USERS. Raises RAISE_APPLICATION_ERROR(-20001, …) if the supplied age is outside the range 5–100, providing a meaningful error message beyond a raw CHECK constraint.

```
-- Attempting to insert age=3 raises:
-- ORA-20001: Invalid age 3 for user "Ghost". Age must be between 5 and 100.
```

9.3 Stored Procedure — recommend_sports_explicit

★ Upgraded from implicit FOR loop to explicit cursor

Key improvements over the original procedure:

- **Explicit named cursor**: CURSOR c_sports IS SELECT … declared in the declaration block; opened, fetched, and closed explicitly.
- **Cursor attributes**: %ROWTYPE for the fetch variable, %NOTFOUND for the EXIT condition, %ISOPEN guard in the EXCEPTION block.
- **Exception handling**: WHEN NO_DATA_FOUND catches an invalid user ID gracefully; WHEN OTHERS closes the cursor if open and re-raises.

```
OPEN c_sports;
LOOP
FETCH c_sports INTO v_rec;
EXIT WHEN c_sports%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('■ ' || v_rec.sport_name || …);
END LOOP;
CLOSE c_sports;
```

9.4 Cursor Summary

Explicit	c_sports	Iterate matched sports for a given user in the stored procedure

10. Transaction Management & Concurrency

Data integrity is maintained through explicit transaction control across all DML operations.

- **COMMIT**: Finalises all INSERT statements after data load (end of 02_data.sql and 04_advanced.sql).
- **SAVEPOINT**: A named savepoint *before_test_insert* is set before a test row is inserted.
- **ROLLBACK TO SAVEPOINT**: The test insert is discarded, demonstrating partial rollback without affecting committed data.

```
SAVEPOINT before_test_insert;
INSERT INTO users VALUES (99, 'Test User', 25, 'Male');
ROLLBACK TO SAVEPOINT before_test_insert; -- discards test row
```

ACID properties are maintained by Oracle's transaction model: Atomicity (COMMIT/ROLLBACK), Consistency (CHECK constraints + triggers), Isolation (Oracle MVCC), and Durability (redo logs on COMMIT). Concurrency control is handled by Oracle's row-level locking and multi-version read consistency.

11. Tools & Technologies Used

- **DBMS:** Oracle Database (Oracle Live SQL — cloud-hosted)
- **Language:** SQL (DDL, DML, DQL, TCL)
- **Language:** PL/SQL (Procedures, Functions, Triggers, Cursors, Exception Handling)
- **Platform:** Windows 11
- **Interface:** Oracle Live SQL (browser-based), SQL*Plus compatible

12. Expected Outcomes

- A fully normalised (3NF) relational database with enforced referential and domain integrity.
- Accurate, reproducible sport recommendations driven by seven matching criteria.

• Efficient data retrieval using indexed JOINs through the recommendation VIEW.
• Reusable PL/SQL function (get_stamina_score) eliminating duplicated CASE logic.
• Automated validation via trigger (trg_validate_user_age) with descriptive error messages.
• Demonstrated explicit cursor lifecycle: OPEN → FETCH → EXIT WHEN %NOTFOUND → CLOSE.
• Demonstrated exception handling: NO_DATA_FOUND, OTHERS with cursor guard and RAISE.
• Analytical insight queries using GROUP BY, HAVING, aggregate functions, and subqueries.
• Demonstrated transaction control with COMMIT, SAVEPOINT, and ROLLBACK.
